CO-1

ASSESSMENT TOOL 2

DATA INTERPRETATION

NAME: ADHIL BIJUKUMAR REGISTRATION NUMBER:192411133

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4308

1: HTTP Response Analysis

A web server returns the following HTTP response header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

Questions:

1. Identify any incorrect or invalid fields in the header.

Incorrect field:

- **Content-Length: -**150
  - Content-Length cannot be negative.
  - It must be a non-negative integer representing the size of the response body in bytes.

Other fields are valid:

- HTTP/1.1 200 OK
- Date
- Content-Type: text/html
- Connection: keep-alive

2. Analyze the issue with Content-Length and explain its impact.

Issue:

- Content-Length is -150, which is invalid because the value must be 0 or greater.

Impact:

- The browser or client cannot determine the correct size of the response.
- The response may fail to load properly.
- The client may terminate the connection or ignore the response.
- It can cause parsing errors and communication problems between the client and server.

3. Determine whether the status code matches the response correctness.

Status Code: 200 OK

Analysis:

- 200 OK means the server successfully processed the request.
- However, because the response header contains an invalid Content-Length, the response is not correctly formatted.
- Therefore, the status code does not fully match the response correctness.
- The server should send a valid response header before returning 200 OK.

4. Suggest corrections for the identified errors.

Corrected HTTP Response Header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

Corrections:

- Change Content-Length: -150 to a valid positive value (for example, 150).
- Ensure the Content-Length exactly matches the number of bytes in the response body.
- Keep the other fields unchanged since they are already valid.

2: DOM Structure Analysis

Consider the following DOM tree representation:

<html>

<head>

<title>Test Page</title>

</head>

<body>

 Hello World

</body>

</html>

Questions:

1. Identify missing or improperly closed elements.

The HTML contains one error:

- The (paragraph) tag is not closed using .
- The tag is closed before the paragraph is explicitly closed.

Error:

Hello World

Correct form:

Hello World

2. Analyze how the browser will interpret this structure.

Modern browsers automatically correct minor HTML errors.

In this case, the browser will:

- Detect that the tag is not closed.
- Automatically insert the missing before the closing .
- Build the DOM as if the HTML were written correctly.

3. Determine the impact on rendering and layout.

Impact:

- Most browsers will display **"**Hello World**"** correctly.
- Automatic correction may cause inconsistent behavior across different browsers.
- Invalid HTML can make debugging difficult.
- It may create problems when JavaScript manipulates the DOM.
- It does not follow HTML standards and reduces code maintainability.

4. Suggest corrections to make the DOM valid.

```
<!DOCTYPE html>
<html>
<head>
   <title>Test Page</title>
</head>
<body>
   <div>
      <p>Hello World</p>
   </div>
</body>
</html>
```

3: GET vs POST in Login System

A login system uses two different methods:

Method URL Example Data Visibility Security Level

GET /login?user=admin&pass=1234 Visible in URL Low

POST /login Hidden in body Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.

GET

- Sends data in the URL as query parameters.
- Data is visible in the browser's address bar.
- Limited amount of data can be sent.

POST

- Sends data in the HTTP request body.
- Data is not visible in the URL.
- Can send larger amounts of data.
- Cannot be bookmarked based on submitted data and is generally not cached by default.

2. Analyze which method is more suitable for login and why.

POST is more suitable for a login system because:

- Username and password are sent in the request body instead of the URL.
- Credentials are not exposed in the browser address bar.
- It provides better privacy than GET.
- It is the standard HTTP method for submitting login forms.

3. Identify security risks in using GET for authentication.

Using GET for login has several risks:

- Username and password are visible in the URL.
- Credentials may be stored in browser history.
- URLs may be saved in bookmarks.
- Credentials can appear in server logs and proxy logs.
- The URL can be accidentally shared, exposing login information.

4. Suggest best practices for secure login implementation.

- Use the POST method for login forms.
- Use HTTPS to encrypt data during transmission.
- Store passwords securely using strong password hashing algorithms.
- Validate user input on both the client and server.
- Use secure session management after successful login.
- Avoid displaying or storing passwords in URLs or logs.

4: HTML Code Inspection

Analyze the following HTML snippet:

<html>

<head>

<title>Sample</title>

</head>

<body>

<h1>Welcome

<p>This is a paragraph

<img src="image.jpg">

</body>

</html>

Questions:

1. Identify missing closing tags.

The following closing tags are missing:

- for the heading.
- for the paragraph.

Missing tags:

</h1>

2. Analyze issues with the <img> tag.

The <img> tag is not incorrect, but it is incomplete.

Issues:

- It is missing the `alt` attribute, which provides alternative text if the image cannot be displayed.
- The alt attribute also improves accessibility for screen readers.

Better version:

<img src="image.jpg" alt="Sample Image">

3. Determine how browsers will handle these errors.

Modern browsers automatically correct minor HTML errors.

- The browser will automatically close the <h1>tag when it encounters the tag.
- The browser will automatically close the tag before the end of the <body>.
- The image will display if image.jpg exists; otherwise, a broken image icon will appear.
- Missing alt text reduces accessibility.

4. Suggest a corrected version of the code.

Correct HTML Code:

<!DOCTYPE html>

<html>

<head>

  <title>Sample</title>

```
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph.</p>
  <img src="image.jpg" alt="Sample Image">
</body>
</html>
```

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.

1. **User enters a URL in the browser**
   The user types a website address (for example, https://www.example.com) into the browser and presses Enter. This tells the browser which website the user wants to access.
2. **DNS lookup occurs**
   The browser sends a request to a DNS (Domain Name System**)** server to convert the domain name (such as www.example.com) into its corresponding IP address (for example, 192.168.1.1). This allows the browser to locate the correct web server.
3. **HTTP request is sent**
   After obtaining the IP address, the browser sends an HTTP request (such as a GET request) to the web server asking for the required webpage or resource.
4. **Server processes the request**
   The web server receives the HTTP request, processes it, retrieves the requested files or data from the server or database if needed, and prepares an appropriate HTTP response.

5. **Response is returned**
   The server sends an HTTP response back to the browser. The response contains a status code (such as 200 OK), response headers, and the requested content (HTML, CSS, JavaScript, images, etc.).

2. Identify where delays can occur.

Delays may occur at:

- **DNS lookup** if DNS resolution is slow.
- **Network transmission** due to slow internet or high latency.
- **Server processing** if the server is busy or the application is complex.
- **Large response size** causing longer download times.
- **Browser rendering** when pages contain large images, many scripts, or complex styles.

3. Analyze the role of DNS in the process.

- DNS (Domain Name System) converts a domain name (for example, `www.example.com`) into its corresponding IP address.
- Without DNS, the browser would not know which server to contact.
- DNS acts like the phone book of the Internet, mapping human-readable names to machine-readable IP addresses.

4. Suggest optimizations to improve performance.

- Use DNS caching to reduce repeated DNS lookups.
- Compress files using Gzip or Brotli.
- Optimize images and other media files.
- Minify HTML, CSS, and JavaScript.
- Use browser caching for static resources.
- Use a Content Delivery Network (CDN) to serve content from servers closer to users.
- Improve server performance and reduce processing time.